
Batch 16 — Approval Management UI

Daniel Macharia <danielmacharia43@gmail.com>
To: <daniel@ics.ac.ke>

Tue, 16 Jun at 08:16

Batch 16 — Approval Management UI

Meat Lovers CIMS powered by YohPal

16A. Update

backend/routes/api.php

Add routes:

```
$router->get('/api/approvals/history', [ApprovalsController::class, 'history']);
$router->post('/api/approvals/{id}/apply', [ApprovalsController::class, 'apply']);
```

16B. Update

backend/app/Controllers/ApprovalsController.php

Add these methods inside the class:

```
public function history(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER']);

    $stmt = Database::connection()->query(
        'SELECT
            ar.*,
            requester.full_name AS requested_by_name,
            approver.full_name AS approved_by_name
        FROM approval_requests ar
        JOIN users requester ON requester.id = ar.requested_by
        LEFT JOIN users approver ON approver.id = ar.approved_by
        ORDER BY ar.created_at DESC'
    );

    Response::success(['approval_history' => $stmt->fetchAll()]);
}

public function apply(int $id): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER']);
    $db = Database::connection();

    $stmt = $db->prepare(
        'SELECT * FROM approval_requests WHERE id = :id LIMIT 1'
    );

    $stmt->execute(['id' => $id]);
    $approval = $stmt->fetch();

    if (!$approval) {
        Response::error('Approval request not found', 404);
    }

    if ($approval['approval_status'] !== 'APPROVED') {
        Response::error('Only approved requests can be applied', 403);
    }

    $requestData = json_decode($approval['request_data'] ?? '{}', true) ?: [];

    if ($approval['approval_type'] === 'ORDER_CANCELLATION') {
        $this->applyOrderCancellation((int) $approval['reference_id']);
    }

    if ($approval['approval_type'] === 'DISCOUNT') {
        $this->applyDiscount((int) $approval['reference_id'], $requestData);
    }
}
```

```

    if ($approval['approval_type'] === 'STOCK_ADJUSTMENT') {
        $this->applyStockAdjustment((int) $approval['reference_id'], $requestData, (int) $user['id']);
    }

    if ($approval['approval_type'] === 'REFUND') {
        Response::error('Refund execution is a placeholder and must be wired in the payment refund phase',
501);
    }

    AuditLogger::log(
        (int) $user['id'],
        'APPROVALS',
        'APPLY_APPROVED_REQUEST',
        $id,
        null,
        $approval
    );

    Response::success([], 'Approved request applied successfully');
}

private function applyOrderCancellation(int $orderId): void
{
    $stmt = Database::connection()->prepare(
        "UPDATE orders
        SET order_status = 'CANCELLED'
        WHERE id = :id"
    );

    $stmt->execute(['id' => $orderId]);
}

private function applyDiscount(int $orderId, array $requestData): void
{
    $discount = (float) ($requestData['discount_amount'] ?? 0);

    $stmt = Database::connection()->prepare(
        'UPDATE orders
        SET discount_amount = :discount,
            total_amount = GREATEST(subtotal - :discount, 0)
        WHERE id = :id'
    );

    $stmt->execute([
        'discount' => $discount,
        'id' => $orderId,
    ]);
}

private function applyStockAdjustment(int $productId, array $requestData, int $userId): void
{
    $quantity = (float) ($requestData['quantity'] ?? 0);

    $stmt = Database::connection()->prepare(
        "INSERT INTO stock_movements
        (product_id, movement_type, quantity, reference_number, notes, user_id)
        VALUES (:product_id, 'ADJUSTMENT', :quantity, :reference_number, :notes, :user_id)"
    );

    $stmt->execute([
        'product_id' => $productId,
        'quantity' => $quantity,
        'reference_number' => $requestData['reference_number'] ?? 'APPROVED-STOCK-ADJUSTMENT',
        'notes' => $requestData['reason'] ?? 'Approved stock adjustment',
        'user_id' => $userId,
    ]);

    $update = Database::connection()->prepare(
        'UPDATE stock_items
        SET current_quantity = current_quantity + :quantity
        WHERE product_id = :product_id'
    );

    $update->execute([
        'quantity' => $quantity,
        'product_id' => $productId,
    ]);
}

```

16C.

apps/admin-web/src/features/approvals/api/approvalsApi.js

```
import apiClient from '../../../lib/apiClient'
```

```

export const approvalsApi = {
  list: async () => {
    const { data } = await apiClient.get('/approvals')
    return data.data.approvals
  },

  history: async () => {
    const { data } = await apiClient.get('/approvals/history')
    return data.data.approval_history
  },

  approve: async (id, payload = {}) => {
    const { data } = await apiClient.post(`/approvals/${id}/approve`, payload)
    return data
  },

  reject: async (id, payload = {}) => {
    const { data } = await apiClient.post(`/approvals/${id}/reject`, payload)
    return data
  },

  apply: async (id) => {
    const { data } = await apiClient.post(`/approvals/${id}/apply`, {})
    return data
  },
}

```

16D.

apps/admin-web/src/features/approvals/pages/ApprovalListPage.jsx

```

import React from 'react'
import { useMutation, useQuery, useQueryClient } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import Button from '../../../components/ui/Button'
import DataTable from '../../../components/ui/DataTable'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { approvalsApi } from '../../../api/approvalsApi'

export default function ApprovalListPage() {
  const qc = useQueryClient()

  const { data, isLoading, error } = useQuery({
    queryKey: ['approval-list'],
    queryFn: approvalsApi.list,
  })

  const approveMutation = useMutation({
    mutationFn: (id) => approvalsApi.approve(id, { approval_notes: 'Approved from admin dashboard' }),
    onSuccess: () => qc.invalidateQueries({ queryKey: ['approval-list'] }),
  })

  const rejectMutation = useMutation({
    mutationFn: (id) => approvalsApi.reject(id, { approval_notes: 'Rejected from admin dashboard' }),
    onSuccess: () => qc.invalidateQueries({ queryKey: ['approval-list'] }),
  })

  const applyMutation = useMutation({
    mutationFn: approvalsApi.apply,
    onSuccess: () => qc.invalidateQueries({ queryKey: ['approval-list'] }),
  })

  const rows = (data || []).map((item) => ({
    ...item,
    actions: (
      <div style={{ display: 'flex', gap: 8, flexWrap: 'wrap' }}>
        <Button onClick={() => approveMutation.mutate(item.id)}>Approve</Button>
        <button onClick={() => rejectMutation.mutate(item.id)} style={rejectButton}>
          Reject
        </button>
        {item.approval_status === 'APPROVED' ? (
          <button onClick={() => applyMutation.mutate(item.id)} style={applyButton}>
            Apply
          </button>
        ) : null}
      </div>
    ),
  }))

  return (
    <Page title="Approvals" subtitle="Approve, reject, and apply operational control requests">

```

```

{approveMutation.error ? <StatusMessage error={approveMutation.error.message} /> : null}
{rejectMutation.error ? <StatusMessage error={rejectMutation.error.message} /> : null}
{applyMutation.error ? <StatusMessage error={applyMutation.error.message} /> : null}

<Card>
  {isLoading ? <LoadingBlock label="Loading approvals..." /> : null}
  {error ? <StatusMessage error={error.message} /> : null}

  {!isLoading && !error ? (
    <DataTable
      columns={[
        { key: 'approval_type', label: 'Type' },
        { key: 'requested_by_name', label: 'Requested By' },
        { key: 'reference_id', label: 'Reference' },
        { key: 'request_reason', label: 'Reason' },
        { key: 'approval_status', label: 'Status' },
        { key: 'created_at', label: 'Created' },
        { key: 'actions', label: 'Actions' },
      ]}
      rows={rows}
    />
  ) : null}
</Card>
</Page>
)
}

const rejectButton = {
  padding: '10px 14px',
  borderRadius: 10,
  border: '1px solid #be123c',
  background: '#fffff2',
  color: '#be123c',
  cursor: 'pointer',
}

const applyButton = {
  padding: '10px 14px',
  borderRadius: 10,
  border: '1px solid #166534',
  background: '#f0fdf4',
  color: '#166534',
  cursor: 'pointer',
}

```

16E.

apps/admin-web/src/features/approvals/pages/ApprovalHistoryPage.jsx

```

import React from 'react'
import { useQuery } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import DataTable from '../../../components/ui/DataTable'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { approvalsApi } from '../../../api/approvalsApi'

export default function ApprovalHistoryPage() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['approval-history'],
    queryFn: approvalsApi.history,
  })

  return (
    <Page title="Approval History" subtitle="Full history of approval requests and management decisions">
      <Card>
        {isLoading ? <LoadingBlock label="Loading approval history..." /> : null}
        {error ? <StatusMessage error={error.message} /> : null}

        {!isLoading && !error ? (
          <DataTable
            columns={[
              { key: 'approval_type', label: 'Type' },
              { key: 'requested_by_name', label: 'Requested By' },
              { key: 'approved_by_name', label: 'Approved/Rejected By' },
              { key: 'reference_id', label: 'Reference' },
              { key: 'approval_status', label: 'Status' },
              { key: 'approval_notes', label: 'Notes' },
              { key: 'approved_at', label: 'Decision Date' },
              { key: 'created_at', label: 'Requested Date' },
            ]}
            rows={data || []}
          />
        ) : null}
      </Card>
    </Page>
  )
}

```

```

    ) : null}
  </Card>
</Page>
)
}

```

16F.

apps/admin-web/src/features/approvals/pages/ApprovalRulesPage.jsx

```

import React from 'react'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'

export default function ApprovalRulesPage() {
  return (
    <Page title="Approval Rules" subtitle="Operational approval enforcement rules">
      <div style={{ display: 'grid', gap: 16 }}>
        <Card title="Order Cancellation">
          Waiters and cashiers may request cancellation, but only manager/admin approval can apply
          cancellation.
        </Card>

        <Card title="Discount">
          Waiters and cashiers may request discount, but only manager/admin approval can apply the discount to
          the order.
        </Card>

        <Card title="Stock Adjustment">
          Storekeepers may request adjustment, but only approved requests can update stock movement and
          balance.
        </Card>

        <Card title="Refund Placeholder">
          Refund approval exists as a control type, but actual refund execution must be wired in the payment
          refund phase.
        </Card>
      </div>
    </Page>
  )
}

```

16G. Update

apps/admin-web/src/app/routes.jsx

Add imports:

```

import ApprovalListPage from '../features/approvals/pages/ApprovalListPage'
import ApprovalHistoryPage from '../features/approvals/pages/ApprovalHistoryPage'
import ApprovalRulesPage from '../features/approvals/pages/ApprovalRulesPage'

```

Add routes inside DashboardLayout:

```

<Route path="/approvals" element={<ApprovalListPage />} />
<Route path="/approvals/history" element={<ApprovalHistoryPage />} />
<Route path="/approvals/rules" element={<ApprovalRulesPage />} />

```

16H. Update

apps/admin-web/src/components/common/SidebarNav.jsx

Add links:

```

['Approvals', '/approvals'],
['Approval History', '/approvals/history'],
['Approval Rules', '/approvals/rules'],

```

16I.

docs/APPROVAL_MANAGEMENT_UI.md

Batch 16 – Approval Management UI

System

Meat Lovers CIMS powered by YohPal

Purpose

This batch gives management a dashboard for approving, rejecting, and applying sensitive operational requests.

Approval Types Supported

1. Order cancellation
2. Discount
3. Stock adjustment
4. Refund placeholder

Pages Added

Approval List

Shows current approval requests and allows:

- approve
- reject
- apply approved request

Approval History

Shows all approval decisions and request history.

Approval Rules

Explains which sensitive actions require management approval.

Apply Logic

Order Cancellation

When an approved order cancellation is applied:

```
```text
orders.order_status = CANCELLED
```

## Discount

When an approved discount is applied:

```
orders.discount_amount = requested discount
orders.total_amount = subtotal - discount
```

## Stock Adjustment

When an approved stock adjustment is applied:

- a stock movement is created
- stock item quantity is updated

## Refund

Refund approval is intentionally not executed yet.

It remains a placeholder until the payment refund phase is built.

## Theft Control Value

This module prevents:

- unauthorized order cancellation
- unauthorized discount
- unauthorized stock adjustment
- unapproved refund execution

---

## # Batch 16 Outcome

Approval management now includes:

- ✅ approval list page
- ✅ approve action
- ✅ reject action
- ✅ approval status history
- ✅ cancellation approval handling
- ✅ discount approval handling

✓ stock adjustment approval handling

✓ refund approval placeholder

# Next Smart Move

Build \*\*Batch 17 – Finance + P&L Deepening Pack\*\*, including:

- expense categories
- income categories
- daily income statement table
- weekly income statement table
- monthly income statement table
- annual income statement table
- food/soft drinks/alcohol profitability split
- cashier settlement reconciliation
- variance detection placeholder